

Progetto Informatica con Laboratorio

Equazione del calore 2D e riordinamento nelle fattorizzazioni Cholesky

Docenti:

Roberto Grossi Leonardo Robol

Studentessa:

Giulia Casti

Anno Accademico 2025/2026

1 Introduzione

Il presente documento descrive lo sviluppo e l'evoluzione software del modulo `HeatEquationSolver`, progettato per la risoluzione numerica dell'equazione del calore 2D e lo studio del riordinamento delle matrici sparse nelle fattorizzazioni di Cholesky.

2 Modulo: `HeatEquationSolver`

Lo sviluppo del solutore procede in modo modulare ed evolutivo. Di seguito vengono dettagliati i singoli sottomoduli.

2.1 Sottomodulo 1: Generazione della griglia del dominio

2.1.1 Specifiche e Funzionalità

Il componente si occupa della discretizzazione spaziale del dominio di calcolo.

- **Input:** Intero N che determina la discretizzazione uniforme del dominio quadrato $[0, 1]^2$.
- **Output:** Griglia 2D rappresentata come insieme di punti.

La griglia è definita come un insieme di punti dove ogni elemento è identificato da una coppia di coordinate (x, y) . La generazione include tutti i punti del dominio, compreso il bordo, con passo spaziale $h = \frac{1}{N}$, per un totale di $(N + 1) \times (N + 1)$ punti.

2.1.2 Strutture Dati

Per la rappresentazione in memoria delle entità coinvolte si adottano le seguenti scelte architetturali:

- **Punto:** Struttura (`struct`) composta da due variabili di tipo `double` per le coordinate x e y .

- **Griglia:** Array dinamico (`vector`) per la gestione della collezione di punti.

2.1.3 Complessità Attesa

La generazione della griglia richiede di iterare su tutti i punti del dominio discretizzato per un totale di $(N + 1)^2$ elementi. La complessità temporale attesa è dunque $O(N^2)$, poiché il calcolo delle coordinate di ciascun punto viene eseguito in tempo costante $O(1)$.

2.1.4 Sviluppo e Scelte Implementative

Il processo di sviluppo è iniziato con la definizione dei tipi globali all'interno di un file header dedicato, `tipiglobali.h`, all'interno del quale sono state definite le strutture dati fondamentali fino ad arrivare al tipo `Nodi`. Inizialmente, come esercizio preparatorio per prendere confidenza con le strutture dati, si è scelto di implementare la generazione di una semplice griglia fisica prima di passare alla topologia del grafo. A tale scopo è stata creata la classe `Generatoregriglia`: si è prima redatto l'header definendo i metodi necessari, e successivamente si è passati all'implementazione nel file `.cpp`, dove la griglia è stata popolata iterando sulle coordinate spaziali tramite due cicli `for` innestati.

2.2 Sottomodulo 2: Gestore File

2.2.1 Specifiche e Funzionalità

Questo componente gestisce le operazioni di I/O su memoria secondaria, occupandosi sia della scrittura (esportazione) sia della lettura (importazione) delle informazioni strutturali e algebriche del problema.

- **Input:** Strutture dati in memoria (griglia, liste di nodi/archi, matrice sparsa A , vettore b) per le operazioni di scrittura; file di testo memorizzati per le operazioni di lettura.

- **Output:** File su memoria secondaria per la fase di esportazione; strutture dati caricate in memoria RAM per la fase di importazione.

Nello specifico, il sottomodulo mette a disposizione le funzionalità necessarie per:

- Generazione, scrittura e lettura del file contenente la lista dei soli punti interni della griglia (escludendo i punti di bordo).
- Generazione, scrittura e lettura del file contenente la lista degli archi che compongono il grafo di adiacenza.
- Generazione, scrittura e lettura del file contenente la lista dei nodi del grafo.
- Generazione, scrittura e lettura del file rappresentante la matrice sparsa dei coefficienti A .
- Generazione, scrittura e lettura del file contenente il vettore dei termini noti (rhs).

2.2.2 Strutture Dati

Per il Gestore File non si rendono necessarie strutture dati complesse aggiuntive in RAM. L'operazione richiede esclusivamente la gestione dei flussi di I/O (stream di input/output su file come `std::ifstream` e `std::ofstream`) per il trasferimento sequenziale dei dati tra la memoria primaria e i file di testo.

2.2.3 Complessità Attesa

Le operazioni di lettura e scrittura su memoria di massa richiedono il passaggio sequenziale attraverso i dati:

- Per i file topologici (nodi e archi): la complessità temporale attesa è $O(V + E) = O(N^2)$.
- Per i file algebrici (matrice A e vettore b): la complessità temporale attesa è proporzionale agli elementi non nulli della matrice e alla dimensione del vettore, ossia $O(|NZ| + |V|) = O(N^2)$.

2.3 Sottomodulo 3: Creazione del grafo di adiacenza

2.3.1 Specifiche e Funzionalità

Il componente si occupa della modellizzazione topologica dei nodi interni mediante la costruzione di un grafo non orientato.

- **Input:** Intero N che determina la discretizzazione uniforme del dominio quadrato $[0, 1]^2$.
- **Output:** Grafo non orientato $G = (V, E)$.

Il grafo modellato soddisfa le seguenti caratteristiche:

- Ogni nodo in V rappresenta un punto interno della griglia (escludendo i nodi di bordo).
- Ogni arco in E rappresenta una relazione di adiacenza orizzontale o verticale tra punti vicini (struttura a 5 punti).
- Ogni nodo possiede coordinate geometriche (x, y) associate.

Le funzionalità richieste per questo sottomodulo includono:

- Generazione dei soli punti interni della griglia.
- Assegnazione di identificatori numerici progressivi ai nodi.
- Costruzione delle relazioni di adiacenza tra nodi adiacenti.
- Esportazione dei dati nei file `coords.txt` e `connectivity.txt`.
- Supporto a successivi riordinamenti dei nodi (es. indicizzazione per algoritmi di fill-in reduction).

2.3.2 Strutture Dati

Per rappresentare la topologia del grafo in modo efficiente si suggeriscono:

- **Liste di adiacenza:** Implementate tramite `std::vector<std::vector<int>>` per garantire una rappresentazione compatta ed efficiente del grafo sparso.
- **Mappatura geometrica/topologica:** Strutture dati ausiliarie (come hash map o array bidimensionali di indicizzazione) per la conversione bidirezionale tra coordinate geometriche (i, j) e identificatori numerici progressivi dei nodi.

2.3.3 Complessità Attesa

La costruzione del grafo e la determinazione delle adiacenze richiedono una complessità temporale attesa pari a $O(|V|) = O((N - 1)^2) = O(N^2)$, poiché il grado di ciascun nodo è limitato superiormente da 4 e l'identificazione dei vicini viene eseguita in tempo costante.

2.3.4 Sviluppo e Scelte Implementative

Sulla base dell'esperienza acquisita con la classe della griglia, si è passati alla modellizzazione del grafo. Seguendo lo stesso pattern di progettazione, è stato scritto prima l'header con la definizione della classe `generatoregrafa`, per poi procedere all'implementazione dei metodi nel file `.cpp`. Durante le primissime fasi di testing e validazione del codice, per facilitare il debugging, l'algoritmo è

stato fatto eseguire utilizzando un valore di discretizzazione fisso e molto ridotto ($N = 4$). L'uso di un "toy problem" ha permesso di verificare manualmente la correttezza della griglia generata e delle corrispettive liste di adiacenza. Una volta confermato il corretto funzionamento della logica di base, la gestione del parametro di input è stata modificata e generalizzata. Il parametro N è stato trasformato da un singolo valore scalare fisso a un costrutto iterabile (un vettore di valori), in quanto requisito fondamentale per la risoluzione dei task successivi (in particolare per il Task 5 e l'analisi sperimentale). Questa transizione ha permesso di automatizzare l'esecuzione del codice per diverse scale del problema (es. $N \in \{32, 64, 128, \dots\}$) in modo sequenziale, senza necessità di ricompilare il codice ad ogni variazione del dominio. Durante lo sviluppo (Task 1), il metodo di calcolo degli archi è stato oggetto di ottimizzazione del codice. Invece di hardcodare e gestire singolarmente ogni direzione per la ricerca dei nodi adiacenti (sopra, sotto, destra, sinistra), si è scelto di implementare un costrutto iterativo (un ciclo for range-based) in grado di scorrere in modo compatto le direzioni (array di 2 int), scritte come stencil a 5 punti. Questa scelta progettuale ha reso il codice molto più pulito, scalabile e facile da mantenere, sebbene non abbia alterato il costo computazionale teorico dell'algoritmo, che rimane saldamente $O(N^2)$.

2.4 Sottomodulo 4: Costruzione dell'ordinamento dei nodi

2.4.1 Specifiche e Funzionalità

Questo sottomodulo implementa la tecnica di riordinamento dei nodi del grafo per ridurre il fenomeno del *fill-in* durante la successiva fattorizzazione di Cholesky della matrice associata.

- **Input:** File `coords.txt` (coordinate dei nodi) e `connectivity.txt` (topologia degli archi del grafo).
- **Output:** File contenente la sequenza ordinata degli identificatori dei nodi (permutazione).

Le funzionalità richieste comprendono:

- Lettura ed elaborazione del grafo di adiacenza a partire dai file d'ingresso.
- Riordinamento dei nodi tramite una procedura ricorsiva basata su strategie di partizionamento (es. Dissecazione Annidata / *Nested Dissection*).
- Identificazione ricorsiva di separatori di vertici che dividono il grafo in sottografi disgiunti, numerando prima i sotto-problemi e infine il separatore.
- Scrittura della permutazione dei nodi ottenuta su file di output.

2.4.2 Strutture Dati

Per supportare la procedura ricorsiva di riordinamento si suggeriscono:

- **Permutazione dei nodi:** Un `std::vector<int>` per memorizzare l'ordine finale dei nodi.
- **Mappatura dei sottografi:** Strutture ausiliarie (come vettori di flag booleani `std::vector<bool>` o sottoliste di vertici) per tracciare i nodi attivi in ciascuna chiamata ricorsiva.
- **Coda / Pila ausiliaria:** Strutture dati di supporto (`std::queue` o `std::vector`) per la ricerca e l'individuazione del separatore di vertici (es. tramite viste BFS/RCM).

2.4.3 Complessità Attesa

La procedura ricorsiva di dissecazione divide il dominio ad ogni livello. La complessità temporale attesa dipende dal costo del partizionamento ad ogni livello di ricorsione:

- Il costo per trovare un separatore su un sottografo con V' nodi è proporzionale alle dimensioni del sottografo stesso, ossia $O(V')$.
- Risolvendo la relazione di ricorrenza $T(V') = 2T(V'/2) + O(V')$, si ottiene una complessità temporale complessiva attesa di $O(|V| \log |V|) = O(N^2 \log N)$.

2.5 Sviluppo e Scelte Implementative

Come per il Task1 è stato innanzitutto creato un header file dedicato, in cui è stata definita la classe `PartizionatoreGrafo`, in cui è stato definito il metodo `eseguiNestedDissectionRicorsiva`. La scelta di esporre un'unica funzione pubblica, rispetto ai tre sotto-passi inizialmente ipotizzati nella progettazione (lettura del grafo, ricerca del separatore, scrittura della permutazione), riflette una decomposizione diversa da quella tratteggiata in fase di specifica iniziale: la lettura dei file è stata delegata al modulo `GestioneFile` (esteso appositamente, si veda oltre), mentre la logica di partizionamento vera e propria è stata isolata in una funzione ausiliaria interna, non esposta nell'header, poiché di interesse esclusivamente implementativo.

La suddivisione di un insieme di nodi nei tre sottoinsiemi V_1 , V_2 e V_S è realizzata dalla funzione ausiliaria `partizionaNodi`, che implementa l'euristica geometrica suggerita dalla traccia: dato l'asse di taglio corrente (x o y , selezionato dal parametro booleano `dividiPerX`), vengono estratte le coordinate distinte dei nodi lungo quell'asse, ordinate, e se ne seleziona il valore mediano $\hat{x}$ (o $\hat{y}$) come piano di taglio. Ogni nodo viene quindi classificato confrontando la propria coordinata con $\hat{x}$: i nodi con coordinata inferiore formano V_1 , quelli con

coordinata superiore V_2 , quelli coincidenti (entro una tolleranza 10^{-9} , necessaria per gestire correttamente gli errori di arrotondamento tipici dell'aritmetica in virgola mobile) formano il separatore V_S .

Per verificare che il separatore individuato geometricamente sia anche un separatore valido dal punto di vista topologico, cioè che V_1 e V_2 non presentino archi diretti tra loro nel grafo, è stata introdotta una funzione di validazione `insiemiSonoDisconnessi`. A tale scopo si è reso necessario estendere il modulo `GestioneFile` con una nuova funzione, `leggiGrafoCompleto`, che combina la lettura di `coords.txt` e `connectivity.txt` per ricostruire non solo le coordinate dei nodi, ma anche le rispettive adiacenze.

Il file `main.cpp` del Task 2 si limita a orchestrare i passaggi precedenti: legge il grafo completo tramite `leggiGrafoCompleto`, avvia la ricorsione sull'intero insieme di nodi con un taglio iniziale lungo l'asse x , e salva la permutazione ottenuta su `ordering.txt` tramite la funzione `generaFileNodiPartizionati` (anch'essa aggiunta a `GestioneFile`), che scrive ciascuna coppia (nuovo indice, identificatore originario) su una riga distinta, nel formato richiesto dalla traccia.

2.6 Sottomodulo 5: Generazione della matrice sparsa

2.6.1 Specifiche e Funzionalità

Questo sottomodulo si occupa dell'assemblaggio del sistema lineare derivante dalla discretizzazione alle differenze finite dell'operatore laplaciano 2D sul dominio quadrato.

- **Input:** Grafo di adiacenza $G = (V, E)$ (o lista dei nodi interni con topologia dei vicini) e vettore di permutazione dei nodi (opzionale per l'ordinamento).
- **Output:** Matrice dei coefficienti A memorizzata in un formato efficiente per matrici sparse.

Le funzionalità richieste includono:

- Costruzione dello stencil a 5 punti per l'operatore laplaciano: elemento diagonale pari a 4, elementi fuori diagonale corrispondenti ai vicini adiacenti pari a -1 .
- Applicazione dell'eventuale permutazione dei nodi ricevuta in input per strutturare la matrice secondo l'ordinamento calcolato nel sottomodulo precedente.
- Rappresentazione della matrice in formato compresso per evitare l'allocazione inutilizzata di zeri.

2.6.2 Strutture Dati

Per rappresentare la matrice sparsa in modo compatto si suggeriscono:

- **Formato CSR (Compressed Sparse Row) o COO (Coordinate Format):** Vettori dinamici `std::vector<double>` per i valori dei coefficienti, `std::vector<int>` per gli indici di colonna e per i puntatori di riga.
- **Mappatura delle permutazioni:** Vettore `std::vector<int>` per tradurre l'indice originario del nodo nell'indice permutato.

2.6.3 Complessità Attesa

Essendo lo stencil limitato a un massimo di 5 elementi non nulli per riga, il numero totale di elementi non nulli nella matrice A è $|NZ| \leq 5|V|$. La complessità temporale attesa per la costruzione della matrice sparsa è quindi proporzionale al numero di nodi interni: $O(|V|) = O((N-1)^2) = O(N^2)$.

2.7 Sviluppo e Scelte Implementative

Il sottomodulo è stato implementato nella classe `GeneratoreMatrice`, con un unico metodo statico, `generaMatriceA(N, kappa, perm)`, coerente con lo schema architetturale già adottato nei Task precedenti.

La costruzione riutilizza lo stesso schema di conversione tra indici logici (i, j) e identificatore progressivo già impiegato nel Task 1, tramite una funzione ausiliaria locale `getOldId`: questo garantisce che l'identificatore "naturale" di ciascun nodo, calcolato qui, coincida sempre con quello prodotto da `GeneratoreGrafo`, senza dover rileggere il grafo generato su file. Tale identificatore naturale viene poi tradotto nell'indice di riga/colonna effettivo tramite il vettore di permutazione `perm`, ricevuto come parametro: questo isola completamente la logica di assemblaggio dello stencil dalla scelta dell'ordinamento, che resta esterna al modulo.

Per ciascun nodo interno viene inserito il coefficiente diagonale $-4\kappa/h^2$, seguito da un coefficiente κ/h^2 per ciascuno dei vicini effettivamente presenti (le stesse quattro condizioni al contorno $i > 1$, $i < N$, $j > 1$, $j < N$ già usate per l'adiacenza nel Task 1, per escludere i vicini che cadrebbero sul bordo del dominio). La permutazione viene applicata sia all'indice di riga sia a quello di colonna di ogni entrata, cosicché la matrice risultante sia già espressa direttamente nell'ordinamento richiesto.

Le entrate non vengono accumulate in una struttura a matrice piena, ma in un vettore di triple (`riga`, `colonna`, `valore`) tramite il tipo `ElementoMatrice`, un

formato coordinato (COO) che rispecchia direttamente il formato di riga richiesto per `A.txt` (`i j A[i,j]`) e che si converte agevolmente in formato sparso compresso nel notebook Python del Task 4.

2.8 Sottomodulo 6: Generazione del termine noto

2.8.1 Specifiche e Funzionalità

Questo sottomodulo genera il vettore dei termini noti b (*rhs*) relativo all'equazione del calore in regime stazionario o al passo temporale della discretizzazione, integrando la sorgente di calore e le condizioni al contorno.

- **Input:** Griglia di punti interni, eventuale funzione sorgente $f(x, y)$, condizioni al contorno di Dirichlet/Neumann definite sul bordo e vettore di permutazione dei nodi.
- **Output:** Vettore dei termini noti b .

Le funzionalità richieste includono:

- Valutazione della funzione sorgente $f(x, y)$ in corrispondenza di ciascun punto interno della griglia.
- Modifica dei valori in prossimità del bordo per incorporare le condizioni al contorno note (es. traslazione al membro destro dei contributi dei nodi di bordo Dirichlet).
- Riordinamento delle componenti del vettore b in accordo con la permutazione dei nodi adottata per la matrice.

2.8.2 Strutture Dati

Per la memorizzazione e la manipolazione del termine noto si suggerisce:

- **Vettore denso:** Un array dinamico `std::vector<double>` di dimensione pari al numero di nodi interni $|V| = (N - 1)^2$.

2.8.3 Complessità Attesa

La valutazione della funzione sorgente e l'applicazione delle condizioni al bordo avvengono puntualmente su ogni nodo interno:

- La valutazione di ciascun elemento e l'eventuale reindicizzazione richiedono tempo costante $O(1)$ per nodo.
- La complessità temporale attesa complessiva è pari a $O(|V|) = O((N - 1)^2) = O(N^2)$.

2.9 Sviluppo e Scelte Implementative

Il sottomodulo è stato implementato nella classe `GeneratoreTermineNoto`, con un unico metodo statico, `generaTermineNoto(N, kappa, f, g, perm)`, specularmente per struttura a `GeneratoreMatrice`.

Come per la matrice A , la posizione di ciascun valore nel vettore risultante è determinata tramite la stessa conversione indici $(i, j) \rightarrow$ identificatore naturale già impiegata nei moduli precedenti, seguita dall'applicazione della permutazione `perm`: questo garantisce che le componenti del vettore b restino coerenti riga per riga con quelle della matrice A , qualunque sia l'ordinamento scelto.

Per ciascun nodo interno il valore viene inizializzato a $-f(x, y)$, la sorgente di calore valutata nelle coordinate del nodo. Se il nodo è adiacente al bordo del dominio (verificato tramite le stesse quattro condizioni $i = 1, i = N, j = 1, j = N$ usate per individuare i vicini mancanti nell'adiacenza del Task 1), viene sottratto anche il contributo $\kappa/h^2 \cdot g(x, y)$ corrispondente, così da incorporare direttamente nel termine noto le condizioni al bordo di Dirichlet, senza introdurre ulteriori incognite. Sia f che g sono ricevute come parametri di tipo `std::function` anziché definite internamente al modulo: questa scelta consente di ridefinirle in `main.cpp` senza modificare l'implementazione, e rende immediata l'estensione opzionale a una condizione al bordo $u_0(x, y) \neq 0$ prevista dalla traccia, che richiederebbe la sola sostituzione della lambda g passata alla chiamata. Nel caso qui trattato, $g(x, y) \equiv 0$.

2.10 Sottomodulo 7: Conversione CSC e Fattorizzazione di Cholesky

2.10.1 Specifiche e Funzionalità

Questo sottomodulo gestisce la conversione della matrice sparsa simmetrica e definita positiva nel formato compresso per colonne e l'esecuzione della fattorizzazione di Cholesky per la risoluzione del sistema lineare $Ax = b$ (oppure $\tilde{A}y = \tilde{b}$ con la matrice permutata).

- **Input:** Matrice sparsa A (in formato COO o CSR) e vettore dei termini noti b .
- **Output:** Matrice triangolare inferiore L della fattorizzazione di Cholesky ($A = LL^T$) in formato CSC e la soluzione del sistema x .

Le funzionalità richieste comprendono:

- Conversione efficiente della struttura della matrice sparsa nel formato CSC (*Compressed Sparse Column*), ottimale per gli algoritmi di fattorizzazione per colonna.

- Calcolo simbolico ed eliminazione per la fattorizzazione di Cholesky $A = LL^T$, sfruttando la simmetria e la struttura sparsa della matrice.
- Risoluzione del sistema tramite sostituzione in avanti ($Ly = b$) e sostituzione all'indietro ($L^T x = y$).
- Valutazione dell'impatto del riordinamento dei nodi sulla riduzione del fenomeno di *fill-in* (confrontando il numero di elementi non nulli di L con e senza riordinamento).

2.10.2 Strutture Dati

Per rappresentare la matrice CSC e la fattorizzazione si utilizzano:

- **Struttura Matrice CSC:** Vettori dinamici `std::vector<double> values` per i valori non nulli, `std::vector<int> row_indices` per gli indici di riga, e `std::vector<int> col_pointers` per i puntatori d'inizio di ciascuna colonna (dimensione $n + 1$).
- **Fattore Cholesky L :** Una seconda struttura CSC dedicata alla memorizzazione della matrice triangolare inferiore risultante L .
- **Vettori di lavoro ausiliari:** Vettori densi `std::vector<double>` per memorizzare i risultati intermedi durante le fasi di eliminazione e di sostituzione avanti/indietro.

2.10.3 Complessità Attesa

L'analisi temporale si articola in due fasi distintive:

- **Conversione in CSC:** L'ordinamento e il raggruppamento per colonna dei $|NZ|$ elementi non nulli richiede tempo $O(|NZ| + |V|) = O(N^2)$.
- **Fattorizzazione di Cholesky:** Su una griglia 2D $N \times N$, la presenza di *fill-in* porta la fattorizzazione ad avere una complessità temporale di $O(N^3) = O(|V|^{3/2})$ in assenza o presenza di opportuno riordinamento (come la *Nested Dissection*), migliorando nettamente rispetto al caso denso $O(|V|^3) = O(N^6)$.
- **Risoluzione triangolare (Forward/Backward substitution):** Proporzionale al numero di elementi non nulli di L , pari a $O(|NZ(L)|) = O(N^2 \log N)$ con riordinamento ottimale.

2.11 Sviluppo e scelte implementative

Una prima versione, precedente alla corretta installazione di `scikit-sparse` sull'ambiente di sviluppo, ha adottato temporaneamente una fattorizzazione di Cholesky densa (`scipy.linalg.cholesky`, previa conversione

esplicita della matrice in un array NumPy pieno). Questa soluzione, pur numericamente corretta e utile per validare la correttezza dell'intera pipeline (lettura dei dati, risoluzione dei sistemi triangolari, calcolo del residuo) per valori di N contenuti, non sfrutta in alcun modo la sparsità della matrice: il costo computazionale e di memoria cresce con l'intera dimensione $N^2 \times N^2$ della matrice densa, rendendola inutilizzabile per i valori di N richiesti dall'analisi sperimentale del Task 5, e vanificando inoltre l'effetto del riordinamento *Nested Dissection* del Task 2, che agisce specificamente sulla struttura sparsa della fattorizzazione.

L'installazione di `scikit-sparse` si è rivelata l'ostacolo principale nello sviluppo di questo sottomodulo: la libreria dipende dalle librerie native SuiteSparse/CHOLMOD, la cui compilazione tramite `pip` non è risultata praticabile sull'ambiente Windows utilizzato. La soluzione adottata è stata la creazione di un ambiente `conda` dedicato (tramite `conda-forge`, che distribuisce SuiteSparse già precompilato), isolato dall'ambiente Python preesistente, la cui versione (Python 3.9) risultava inferiore al requisito minimo della libreria (Python ≥ 3.10).

Una volta resa disponibile la libreria, un primo tentativo di adattare il codice suggerito dalla traccia ha prodotto un errore di esecuzione, causato da un cambiamento nell'interfaccia della funzione `cholesky()` introdotto nella versione più recente di `scikit-sparse`: la funzione non restituisce più un oggetto `Factor` dotato di un metodo `L()`, bensì la matrice fattorizzata direttamente (o, se viene specificato un ordinamento, una tupla contenente la matrice e il vettore di permutazione). La correzione è consistita nell'aggiungere il parametro `lower=True`, non presente nello snippet originale ma necessario per ottenere la fattorizzazione L triangolare inferiore anziché la sua trasposta R (restituita per default), mantenendo per il resto la struttura del codice suggerito dalla traccia (`order="natural"`, accesso tramite indicizzazione `factor[0]`).

Un'ultima criticità è emersa solo in fase di analisi sperimentale su valori di N crescenti (Task 5): un errore di fattorizzazione (matrice non definita positiva) che, ad un'analisi più approfondita, non era riconducibile alla fattorizzazione in sé, ma a un disallineamento nei percorsi dei file intermedi tra i Task precedenti, che impediva la corretta rigenerazione di `coords.txt` e `connectivity.txt` per valori di N diversi da quello di un'esecuzione precedente. Risolto tale disallineamento, la fattorizzazione sparsa risulta corretta e stabile per l'intero intervallo di N testato.

2.12 Sottomodulo 8: Analisi sperimentale e confronto delle prestazioni

2.12.1 Specifiche e Funzionalità

L'ultimo sottomodulo si occupa dell'integrazione del flusso di calcolo all'interno di un Notebook interattivo per la valutazione delle prestazioni empiriche del solutore e la visualizzazione grafica dei risultati.

- **Input:** Sequenza di valori di discretizzazione $N = 32, 64, 128, 256$.
- **Output:** Grafici delle prestazioni temporali e di memoria (*fill-in*), grafici della soluzione numerica $u(x, y)$ e mappe di radura (*sparsity patterns*) delle matrici.

Le funzionalità richieste comprendono:

- Esecuzione della risoluzione del sistema lineare per i differenti valori di N specificati.
- Confronto sistematico dei tempi di calcolo della fattorizzazione e del numero di entrate non-zero (NZ) richieste dal fattore L , valutati sia con l'ordinamento naturale (lessicografico) sia con il riordinamento proposto (*Nested Dissection*).
- Generazione dei plot della soluzione calcolata $u(x, y)$ sul dominio 2D nel Notebook.
- Visualizzazione grafica della struttura di radura della matrice dei coefficienti A e del fattore di Cholesky L (prima e dopo il riordinamento) mediante la funzione `matplotlib.pyplot.spy`.

2.12.2 Strutture Dati

Per l'analisi sperimentale si impiegano:

- **Array e DataFrame (Python/Jupyter):** Strutture dati tipo `numpy.ndarray` o `pandas.DataFrame` per la memorizzazione e organizzazione dei tempi di esecuzione, del conteggio dei non-zeri (NZ) e degli errori residui.
- **Oggetti grafici Matplotlib:** Matrici e griglie di puntatori gestite da librerie esterne per la rappresentazione visiva dei dati.

2.12.3 Complessità Attesa

L'esecuzione dell'analisi benchmark per l'insieme dei valori $N \in \{32, \dots, 256\}$ richiede:

- Una complessità temporale dominante determinata dal valore massimo $N_{max} = 256$. La complessità complessiva delle simulazioni è $O(\sum N^3) = O(N_{max}^3)$.

- La generazione visiva dei pattern di radura tramite `spy` e la costruzione dei plot 2D/3D introducono un costo proporzionale unicamente al numero di elementi da tracciare, ovvero $O(|NZ(L)|)$ per ogni configurazione testata.

2.13 Sviluppo e scelte implementative

L'implementazione del Task 5 ha richiesto un'intensa e strutturata fase di debugging e validazione dei dati. La primissima stesura del codice presentava infatti alcune anomalie: oltre al già citato problema della libreria di Cholesky affrontato nel Task 4, che appiattiva i vantaggi della sparsità, si è riscontrato un disallineamento nei file di output generati a causa di una mancata e corretta ricompilazione dei binari C++ e di conflitti nei percorsi di esecuzione.

Esami di Diagnostica e Validazione Per isolare e correggere i problemi di compilazione e le anomalie computazionali riscontrate (in particolare gli errori di I/O sui file e l'iniziale anomalia sulla fattorizzazione di Cholesky), è stata introdotta una suite sequenziale di esami di diagnostica all'interno del notebook Python. Ogni test è stato progettato per validare uno specifico passaggio della pipeline:

- **1. Controllo preliminare delle dimensioni dei file di output**
 - **Cosa fa:** Verifica la dimensione effettiva dei file `rhs.txt` e `ordering.txt`.
 - **Perché lo fa:** Per individuare tempestivamente l'errore di mancata compilazione o esecuzione dei task C++, garantendo che Python non stesse lavorando su dati obsoleti generati in esecuzioni precedenti.
 - **Come lo fa:** Caricando i file in memoria tramite `np.loadtxt` e stampandone la lunghezza con la funzione `len()`.
- **2. Test di scalabilità spaziale (Ordinamento Naturale per $N = 32$ e $N = 64$)**
 - **Cosa fa:** Ispeziona la generazione dei file `coords.txt`, `ordering.txt`, `A.txt` e `rhs.txt` impostando `usaOrdering=0`.
 - **Perché lo fa:** Per confermare che i sottomoduli C++ scalassero correttamente le dimensioni delle strutture dati all'aumentare dei nodi interni del dominio.
 - **Come lo fa:** Eseguendo i Task 1, 2 e 3 passandogli progressivamente i parametri $N = 32$ e $N = 64$, e ispezionando la `shape` degli array risultanti.

- **3. Test di scalabilità spaziale (Nested Dissection per $N = 32$ e $N = 64$)**

- **Cosa fa:** Ripete i test di generazione strutturale e algebrica impostando il flag `usaOrdering=1`.
- **Perché lo fa:** Per accertare che il parametro di selezione dell'ordinamento venisse recepito dal backend C++ e innescasse la corretta logica di assemblaggio permutato per la matrice e il termine noto.
- **Come lo fa:** Analogamente al test precedente, alterando unicamente il flag passato via linea di comando a `esecuzioneTask3`.

- **4. Validazione matematica della permutazione topologica**

- **Cosa fa:** Analizza le proprietà algebriche dei vettori estratti dal file `ordering.txt` (nuovi indici e vecchi indici).
- **Perché lo fa:** Per accertarsi che il separatore di vertici calcolato nel Task 2 producesse una permutazione rigorosa, ossia una biiezione perfetta senza saltare alcun nodo interno né creare duplicati o fuori limite.
- **Come lo fa:** Estrapolando il valore minimo (`min()`), massimo (`max()`) e la conta degli elementi unici (`len(np.unique())`) di entrambe le colonne del file di ordinamento.

- **5. Controllo di coerenza strutturale tra matrici (Naturale vs Ordinato)**

- **Cosa fa:** Confronta in parallelo `A_naturale` con `A_ordered` e `rhs_naturale` con `rhs_ordered`.
- **Perché lo fa:** Per garantire matematicamente che l'ordinamento di tipo Nested Dissection non alterasse il contenuto del sistema lineare, ma producesse la medesima matrice e lo stesso termine noto unicamente permutati (stesso numero di incognite e di equazioni).
- **Come lo fa:** Eseguendo la pipeline completa prima per `usaOrdering=0` e poi per `usaOrdering=1` (con $N = 64$), stampandone le rispettive proprietà `shape` a schermo e verificandone l'uguaglianza.

- **6. Test di robustezza della fattorizzazione di Cholesky**

- **Cosa fa:** Mette alla prova l'algoritmo di fattorizzazione `my_cholesky` sulle due differenti versioni del sistema lineare.

- **Perché lo fa:** Per confermare che la matrice risultasse definita positiva e strutturalmente integra per l'algoritmo `sksparse.cholmod` in ambedue i casi, scongiurando eventuali fallimenti numerici legati a riordinamenti errati.
- **Come lo fa:** Passando sequenzialmente `A_nat` e `A_ord` alla funzione e inserendo due istruzioni `print("OK")` a valle delle chiamate, per confermarne l'esecuzione al netto di eventuali eccezioni *MatrixNotPositiveDefinite*.

Correzione e Ottimizzazione del Rendering 3D

L'ultimo e fondamentale intervento ha riguardato il plot della piastra termica. L'implementazione originaria impiegava il costrutto `np.linspace(0, 1, N)` e una *list comprehension* che sfruttava le variabili x e y come indici:

```
x = np.linspace(0,1,N)
y = np.linspace(0,1,N)
plottedU = [[soluzioneU[(N,0)][x*N+y] for x in range(N)] for y in range(N)]
```

Questa logica era concettualmente e matematicamente errata per due motivi:

- 1. Errore nel Dominio Spaziale:** La funzione `linspace(0,1,N)` campiona N punti estremi inclusi. Tuttavia, il sistema lineare calcola la temperatura *esclusivamente* sui nodi interni della griglia, il cui passo spaziale corretto è $h = \frac{1}{N+1}$. Di conseguenza, le coordinate fisiche reali degli N nodi interni lungo un asse non partono da 0, ma spaziano da $\frac{1}{N+1}$ a $\frac{N}{N+1}$.
- 2. Shadowing e Fuori Limite (Out of Bounds):** L'uso delle stesse lettere (x ed y) sia per l'array spaziale che come iteratori per il ciclo `range(N)` generava ambiguità a livello di scope delle variabili e un calcolo impreciso o fuori limite dell'indice `[x*N+y]`.

Il blocco è stato pertanto riscritto e normalizzato. Per generare l'esatta griglia di coordinate `meshgrid` dei punti interni, ci si è avvalsi della formula `np.arange(1, N+1) / (N+1)`. L'estrazione della soluzione esatta u è stata poi disaccoppiata dall'array spaziale, adottando iteratori interi puri (i e j) per mappare in modo esatto la stringa monodimensionale `u_naturale[i*N + j]` sulla matrice topologica bidimensionale Z , eliminando i crash di indicizzazione e restituendo una proiezione `plot_surface` coerente con il dominio matematico studiato.